PONTIS

Spécification Technique — PaaS Propriétaire

v1.2 — 2025

Mise à jour : ajout table refresh_tokens au modèle de données

1. Vision & Objectifs

Pontis est une plateforme PaaS (Platform as a Service) auto-hébergée qui centralise le déploiement, la gestion et le CI/CD de tous vos projets web. L'objectif est de reproduire l'expérience Netlify / Vercel / Render / Heroku en gardant une souveraineté totale sur les données, l'infrastructure et les coûts.

Principes fondateurs

- Souveraineté : tout tourne sur votre propre serveur, aucune dépendance cloud externe
- Simplicité : déployer un projet ne doit prendre que quelques clics
- Automatisation : le CI/CD est déclenché automatiquement à chaque push GitLab
- Isolation : chaque projet vit dans son propre container Docker
- Sécurité : HTTPS automatique via Let's Encrypt pour chaque sous-domaine

Domaine de production validé

Toutes les applications sont déployées sous *.app.ongoua.pro Exemple : monporfolio.app.ongoua.pro DNS hébergé sur Porkbun — challenge DNS-01 via API Porkbun

2. Architecture Technique

2.1 Vue d'ensemble

Composant	Rôle
Dashboard	Interface web React/Next.js pour gérer projets & déploiements
API Backend	Serveur Node.js (Fastify) — cerveau de la plateforme
PostgreSQL	Base de données principale (utilisateurs, projets, déploiements)
Docker Engine	Exécution et isolation de chaque projet déployé
Traefik v3	Reverse proxy dynamique + ACME DNS-01 (Let's Encrypt) automatique
GitLab CE	Source des projets + déclencheur CI/CD via webhooks
Redis	Queues de build asynchrones + cache de sessions

2.2 Stack Technologique

Couche	Technologie
Couche	Technologie

Frontend	Next.js 14 (App Router) + Tailwind CSS
Backend API	Node.js 20 + Fastify + Zod (validation)
Base de données	PostgreSQL 16 + Prisma ORM
Cache & Queues	Redis 7 + BullMQ
Containers	Docker Engine 29.x + Docker SDK (node)
Reverse Proxy	Traefik v3 (config dynamique via labels Docker)
SSL	Let's Encrypt via ACME DNS-01 (certificat wildcard *.app.ongoua.pro)
DNS Provider	Porkbun (API native supportée par Traefik)
Auth	Passport.js (OAuth2 GitLab) + JWT (email/password)
Build	Nixpacks (détection automatique du runtime)
DNS	Entrée wildcard *.app.ongoua.pro → IP du serveur

3. Infrastructure & Configuration Serveur

3.1 Environnement de production

Paramètre	Valeur validée
Paramètre	Valeur validée
OS	Debian 6.1.162-1 (kernel 6.1.0-43-cloud-amd64)
Docker	Docker Engine 29.3.0
Domaine	ongoua.pro (Porkbun)
Pattern sous-domaines	*.app.ongoua.pro
IP serveur	À documenter selon VM de production
Certificat SSL	Wildcard *.app.ongoua.pro via Let's Encrypt DNS-01

3.2 Configuration DNS (Porkbun)

Une seule entrée DNS wildcard suffit pour couvrir tous les projets déployés :

```
Type : A
Host : *.app
Value : <IP_DU_SERVEUR>
TTL  : 300
```

Important — Wildcard impose DNS-01

Un certificat wildcard (*.app.ongoua.pro) ne peut pas être émis via le challenge HTTP-01. Traefik doit utiliser le challenge DNS-01 en appelant l'API Porkbun pour créer/supprimer des enregistrements TXT de validation.

3.3 Configuration Traefik v3

Fichier traefik.yml de référence validé en production :

```
entryPoints:
  web:
    address: ":80"
    http:
      redirections:
        entryPoint:
          to: websecure
          scheme: https
  websecure:
    address: ":443"
```

```
providers:
  docker:
    exposedByDefault: false   # Sécurité : ignore les containers sans label

api:
  dashboard: true
  insecure: true              # Dashboard sur :8080 (désactiver en prod)

certificatesResolvers:
  letsencrypt:
    acme:
      email: contact@ongoua.pro
      storage: /certs/acme.json
      dnsChallenge:
        provider: porkbun
        resolvers:
          - "1.1.1.1:53"
          - "8.8.8.8:53"
```

3.4 Docker Compose — Stack Principale Pontis

Pontis tourne dans un Docker Compose regroupant ses services internes. Les clés API Porkbun sont passées en variables d'environnement au container Traefik :

```
networks:
  pontis_network:
    name: pontis_network      # nom fixe, référencé par toutes les apps

services:
  traefik:
    image: traefik:v3
    restart: always
    ports:
      - "80:80"
      - "443:443"
      - "8080:8080"
    environment:
      - PORKBUN_API_KEY=<pk1_...>
      - PORKBUN_SECRET_API_KEY=<sk1_...>
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - ./traefik.yml:/traefik.yml:ro
      - traefik_certs:/certs
    networks:
      - pontis_network

  api:
    build: ./api
    restart: always
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
```

```
    networks:
      - pontis_network
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.api.rule=Host(`api.app.ongoua.pro`)"
      - "traefik.http.routers.api.tls.certresolver=letsencrypt"
      - "traefik.http.services.api.loadbalancer.server.port=3001"

  postgres:
    image: postgres:16
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - pontis_network
    # Pas de labels Traefik — jamais exposé à l'extérieur

  redis:
    image: redis:7-alpine
    restart: always
    networks:
      - pontis_network

volumes:
  postgres_data:
  traefik_certs:
```

4. Structure d'une Application Déployée

Chaque application gérée par Pontis suit une structure standardisée à trois fichiers. C'est le template que le Worker génère automatiquement pour chaque projet.

4.1 Structure de fichiers

```
projet-slug/
├── docker-compose.yml
├── Dockerfile
└── server.js   (ou tout autre point d'entrée)
```

4.2 docker-compose.yml type

Le service est toujours nommé app. L'unicité est portée par le slug dans les labels Traefik, pas par le nom du service.

```
networks:
  pontis_network:
    external: true   # réseau Traefik existant, ne pas recréer

services:
  app:
    image: registry.local/<slug>:<commit-sha>
    networks:
      - pontis_network
    labels:
      - "traefik.enable=true"
      - "traefik.http.routers.<slug>.rule=Host(`<slug>.app.ongoua.pro`)"
      - "traefik.http.routers.<slug>.entrypoints=websecure"
      - "traefik.http.routers.<slug>.tls=true"
      - "traefik.http.routers.<slug>.tls.certresolver=letsencrypt"
      - "traefik.http.services.<slug>.loadbalancer.server.port=3000"
```

Règle d'unicité

Le nom du service (app) est identique pour tous les projets — il est local au Compose. Seul le slug dans les labels Traefik doit être unique globalement. Ce slug est généré depuis le nom du projet et vérifié en base PostgreSQL.

4.3 Réseau Docker Bridge — pontis_network

Tous les containers (Pontis et projets) partagent le même réseau bridge nommé pontis_network. Ce réseau est créé une seule fois par le Compose principal de Pontis et référencé en external: true par chaque application.

Service	Exposition réseau
Service	Accessible depuis internet
Traefik	Oui — ports 80/443
API Pontis	Oui — via Traefik
Dashboard Pontis	Oui — via Traefik
PostgreSQL	Non — réseau interne uniquement
Redis	Non — réseau interne uniquement
Containers projets	Oui — via Traefik uniquement

Socket Docker — point de sécurité critique

Monter /var/run/docker.sock dans un container lui donne un accès root complet sur le serveur hôte. Ce montage est réservé à Traefik et à l'API Pontis (Worker). Ne jamais l'exposer à un container d'un projet utilisateur.

5. Déploiement Zero Downtime (Blue/Green)

La version naïve (stop ancien → start nouveau) crée une fenêtre d'indisponibilité. Pontis implémente un déploiement blue/green pour garantir zéro interruption de service.

5.1 Séquence de déploiement

Étape	Action
Étape	Action
1	Démarrer le nouveau container sur un nouveau port (N+1)
2	Health check loop — attendre que /health répond HTTP 200
3	Basculer les labels Traefik vers le nouveau container
4	Arrêter l'ancien container (port N)
Rollback	Si health check timeout (30s) → annuler, laisser l'ancien tourner, marquer failed

5.2 Implémentation du health check

```
async function waitForHealthy(port, timeout = 30000) {
  const start = Date.now();
  while (Date.now() - start < timeout) {
    try {
      const res = await fetch(`http://localhost:${port}/health`);
      if (res.ok) return true;
    } catch {}
    await new Promise(r => setTimeout(r, 500));
  }
  throw new Error('Health check timeout — rollback');
}
```

Contrainte blue/green

Le serveur doit supporter deux containers du même projet actifs simultanément pendant quelques secondes. Pour la majorité des apps web, la RAM supplémentaire est négligeable.

6. Modules Fonctionnels

6.1 Authentification

- GitLab CE SSO via OAuth2 (Passport.js)
- Email + Mot de passe (JWT RS256, bcrypt, refresh token httpOnly)

6.2 Build Pipeline

Étape	Description
Étape	Description
1. Clone	Clone du repo GitLab CE via API
2. Détection	Nixpacks analyse le projet (package.json, requirements.txt...)
3. Build	Construction de l'image Docker
4. Push	Image poussée dans le registry Docker local
5. Deploy	Blue/green deployment + mise à jour Traefik

6.3 Attribution de Ports

- Plage réservée : 10000–60000
- PostgreSQL maintient la table ports avec les allocations en cours
- Le Worker interroge la base pour trouver le prochain port libre
- Port + container ID enregistrés en base, liés au projet

6.4 Variables d'Environnement

- Saisies via le dashboard, chiffrées AES-256-GCM en base
- Injectées dans le container au démarrage via --env-file
- Ne jamais apparaître dans les logs ou l'UI après saisie

7. Modèle de Données

Table	Champs principaux
Table	Champs principaux
users	id, email, password_hash, gitlab_id, gitlab_token, created_at
projects	id, user_id, name, slug, type (git|static), domain, port, status

deployments	id, project_id, commit_sha, status (pending|building|success|failed), logs, created_at
env_vars	id, project_id, key, value_encrypted
ports	id, port_number, project_id, allocated_at
refresh_tokens	id, user_id, token_hash, expires_at, revoked_at, created_at

8. Roadmap — Plan de Développement

Phase 1 — Fondation Infrastructure ✓ Validée

- Provisionner VM Debian + Docker Engine 29.x
- Configurer Traefik v3 avec challenge ACME DNS-01 (Porkbun)
- Configurer entrée DNS wildcard *.app.ongoua.pro → IP serveur
- Déployer PostgreSQL + Redis dans Docker Compose
- Valider routing HTTPS d'un container test (mbolo.app.ongoua.pro)
- Valider pattern app indépendante avec external: true sur pontis_network

Phase 1 complétée ✓

Infrastructure validée en conditions réelles : • https://mbolo.app.ongoua.pro → Mbolo Pontis • Certificat wildcard *.app.ongoua.pro actif • Traefik détecte les containers dynamiquement • Pattern docker-compose app indépendante validé

Phase 2 — Authentification

- Initialiser monorepo : /api (Fastify), /web (Next.js), /worker (BullMQ)
- Créer schéma Prisma initial (users, projects, deployments, ports, refresh_tokens)
- Implémenter JWT email/password avec bcrypt
- Implémenter OAuth2 GitLab CE (Passport.js)
- Créer pages Login/Register dans le dashboard
- Protéger les routes API avec middleware JWT

Phase 3 — Gestion de Projets (Sites Statiques)

- API CRUD projets (création, liste, suppression)
- Upload de fichiers statiques (multer + extraction archive)
- Génération de slug + vérification d'unicité en base
- Lancement container Nginx + attribution port + labels Traefik
- Certificat SSL automatique via Traefik DNS-01

Phase 4 — Intégration GitLab CE & Build Pipeline

- Intégration API GitLab CE (liste des repos accessibles)
- Intégration Nixpacks pour détection runtime et build image
- Registry Docker local pour stockage des images
- Déploiement blue/green avec health check
- Logs de build en temps réel (WebSocket)

Phase 5 — CI/CD Automatique

- Configuration webhook GitLab CE (signature HMAC)
- Endpoint API pour recevoir les webhooks de push
- Queue jobs BullMQ pour builds asynchrones
- Worker de build en arrière-plan
- Historique des déploiements + notifications WebSocket

Phase 6 — Production & Observabilité

- Variables d'environnement chiffrées AES-256
- Logs containers accessibles depuis le dashboard
- Métriques basiques (CPU, RAM, uptime par container)
- Rollback vers déploiement précédent
- Isolation réseau par projet (bridge dédié par app)

9. Considérations de Sécurité

- Traefik est le seul composant exposé sur internet (ports 80/443)
- exposedByDefault: false dans traefik.yml — seuls les containers avec traefik.enable=true sont routés
- Socket Docker (:ro) monté uniquement dans Traefik et l'API Worker
- PostgreSQL et Redis accessibles uniquement via pontis_network, jamais depuis l'extérieur
- JWT signés RS256 — access token 15min + refresh token 7j en cookie httpOnly/Secure
- Variables d'environnement chiffrées AES-256-GCM en base
- Clés API Porkbun passées en variables d'environnement, jamais committées
- Chaque container projet tourne en mode non-root

Pontis — Spécification Technique v1.2

Document confidentiel — Usage interne